Name: Rithika

Task: Inventory Management using MySQL

SQL Code

```
DROP DATABASE IF EXISTS quickstartdb;
CREATE DATABASE quickstartdb;
USE quickstartdb;

-- Create a table and insert rows
DROP TABLE IF EXISTS inventory;
CREATE TABLE inventory (
  id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(50),
  quantity INTEGER
);

INSERT INTO inventory (name, quantity) VALUES ('banana', 150);
INSERT INTO inventory (name, quantity) VALUES ('orange', 154);
INSERT INTO inventory (name, quantity) VALUES ('apple', 100);

-- Read
SELECT * FROM inventory;

-- Update
UPDATE inventory SET quantity = 200 WHERE id = 1;
SELECT * FROM inventory;

-- Delete
DELETE FROM inventory WHERE id = 2;
SELECT * FROM inventory;
```

Code Explanation

1. Database setup

DROP DATABASE IF EXISTS quickstartdb; removes any old copy of the database so the script can run cleanly from scratch. CREATE DATABASE quickstartdb; makes a fresh database, and USE quickstartdb; tells MySQL that all following commands should run inside it.

2. Table creation

DROP TABLE IF EXISTS inventory; clears out any old inventory table. CREATE TABLE inventory defines three columns: id (an integer that auto-increments and serves as the primary key, so every row gets a unique identifier automatically), name (a text field up to 50 characters for the item name), and quantity (an integer for stock count).

3. Inserting records

Three INSERT INTO statements add starting inventory: banana with quantity 150, orange with 154, and apple with 100. Since id is AUTO_INCREMENT, MySQL assigns ids 1, 2, and 3 automatically in the order the rows are inserted.

4. Reading records

SELECT * FROM inventory; retrieves and displays every column and row currently in the table, which is used here to confirm the inserts worked.

5. Updating a record

UPDATE inventory SET quantity = 200 WHERE id = 1; changes banana's quantity from 150 to 200. The WHERE clause is important -- it limits the change to only the row where id equals 1, so no other rows are affected.

6. Deleting a record

DELETE FROM inventory WHERE id = 2; removes the row with id 2, which is the orange record. Again, WHERE restricts the delete to just that one row.

7. Final state

After these operations, two records remain: banana with quantity 200 and apple with quantity 100. The orange record was removed, and its slot (id 2) is not reused since the auto-increment counter keeps advancing.

Output

After inserting the records:

id	name	quantity
1	banana	150
2	orange	154
3	apple	100

After updating banana's quantity to 200:

id	name	quantity
1	banana	200
2	orange	154
3	apple	100

After deleting orange:

id	name	quantity
1	banana	200
3	apple	100

Result

S.No	Query Executed	Result
1	INSERT INTO inventory VALUES ('banana', 150)	1 row affected
2	INSERT INTO inventory VALUES ('orange', 154)	1 row affected
3	INSERT INTO inventory VALUES ('apple', 100)	1 row affected
4	SELECT * FROM inventory	3 rows returned
5	UPDATE inventory SET quantity=200 WHERE id=1	1 row affected
6	SELECT * FROM inventory	3 rows returned
7	DELETE FROM inventory WHERE id=2	1 row affected
8	SELECT * FROM inventory	2 rows returned

The script successfully created the inventory table, inserted three records, updated the quantity of banana, and deleted the orange record. The final table contains 2 records: banana (200) and apple (100).